

Arquitectura de microservicios

UnoArena — defensa final

Grupo 5

El dominio

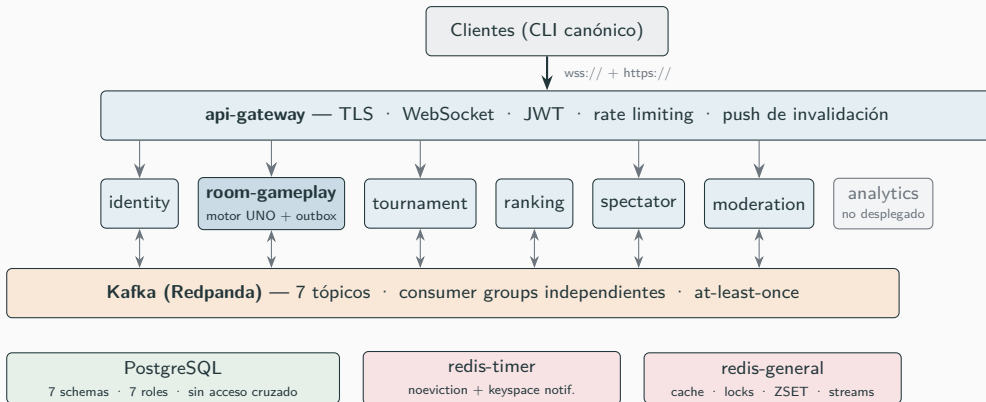
- UNO multijugador global, en tiempo real
- **Quick Play**: salas de 2–10 por matchmaking
- **Torneos de eliminación**: rondas Bo3, top 3 avanza

La escala de diseño

- Hasta **1.000.000** de jugadores por torneo
- Ronda 1 $\Rightarrow$ **~ 100.000 salas simultáneas**
- Espectadores 10:1 $\Rightarrow$ ~ 11 M conexiones WS

El pico de la ronda 1 es el problema: casi todas las decisiones de arquitectura se justifican contra ese surge.

Arquitectura final



1 bounded context = 1 servicio = 1 schema con rol propio · despliegue completo con `deploy-all.sh` sobre kind

Bounded contexts: por qué estos siete

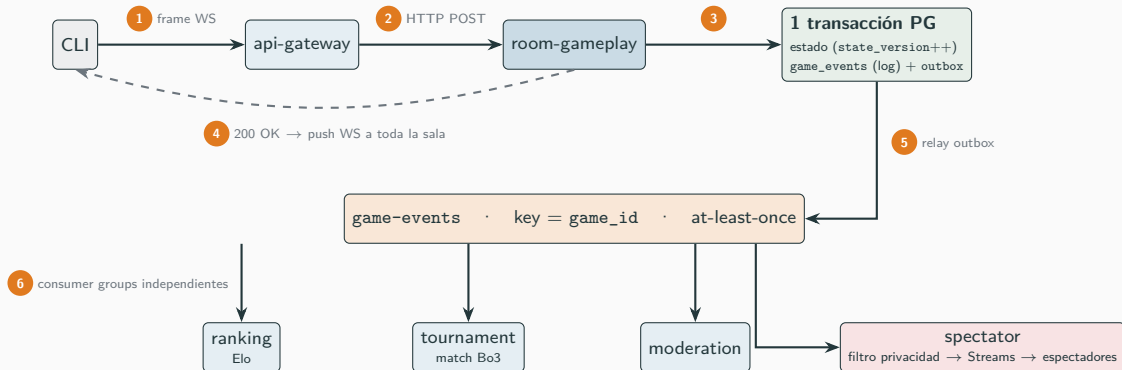
- **Room Gameplay:** núcleo write-heavy, row-lock por juego
- **Tournament:** orquestación Bo3, otro vocabulario (“forfeit” = eliminación permanente, no salir del juego)
- **Ranking:** única autoridad de Elo, reversión auditada
- **Spectator:** proyección read-only + ACL de privacidad
- **Identity:** sesión única, upstream de todos
- **Moderation:** audit-log, comandos correctivos
- **Analytics:** diseñado, no desplegado (ADR-009)

Cada contexto tiene consistencia y ciclo de vida de datos distintos $\Rightarrow$ sin base compartida.

Lo que evaluamos y descartamos

- BFF de juego separado del gateway
 $\rightarrow$ duplica JWT, rate limit e invalidación (ADR-007)
- Matchmaking como servicio propio
 $\rightarrow$ cada modo ya tiene dueño natural
- Plegar Analytics en Ranking
 $\rightarrow$ erosiona la frontera de autoridad

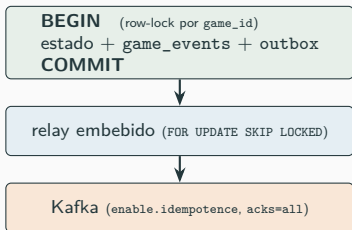
Flujo completo: jugar una carta



- El jugador ve su resultado **después del COMMIT, sin esperar a Kafka** → hot path < 100 ms (P99 < 200 ms).
- WebSocket y no SSE/polling: full-duplex en 1 conexión y *close frame* server-side para la sesión única (ADR-001).
- Conflicto de versión ⇒ 409 `stale_state_version` + reconciliación por el log — concurrencia optimista, sin locks distribuidos.

Decisión: outbox transaccional (log-before-broadcast)

Regla: nadie observa un cambio de estado antes de que esté en el log durable.



Alternativas evaluadas

✗ Dual-write DB + Kafka:

crash entre escrituras = evento perdido o
broadcast falso

✗ Event sourcing puro:

rehidratar 400–1000 eventos por comando =
latencia

✓ Outbox: atomicidad gratis,

estado JSONB → comando O(1)

El traceparent del comando viaja *dentro de la fila de outbox* → la traza cruza Kafka (se ve en Jaeger).

Decisión: Kafka como backbone — tópicos, keys y particiones

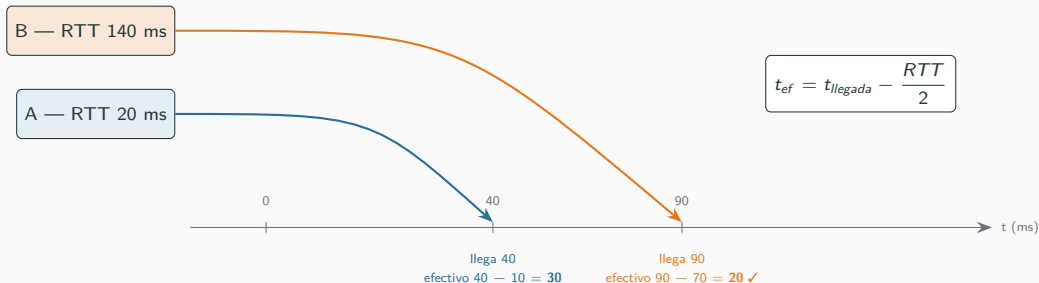
¿Por qué Kafka? Consumer groups con lag independiente (aislar el burst), orden por partición, replay durable. RabbitMQ: sin fan-out con lag propio ni replay. Redis Streams: sin partition routing, memoria acotada.

Tópico	Partition key	Particiones (diseño / demo)	La key es la unidad de orden de...
game-events	game_id	100+ / 8	cada partida (spectator, torneo)
tournament-kickoff	room_id	100+ / 8	fan-out del surge
tournament-events	tournament_id	20 / 3	el bracket
identity-events	player_id	20 / 3	la sesión de un jugador
ranking-events	player_id	20 / 3	el Elo de un jugador
moderation-events	player_id/game_id	10 / 3	el sujeto del evento

- El tipo de evento va como **header**, no en la key ni en tópicos por tipo.
- game-events domina (~300 K ev/s pico) → se lleva las particiones; consumers escalan hasta ese límite.
- Demo = scale-down documentado: misma semántica, un broker.

Decisión: fairness entre regiones (lag compensation)

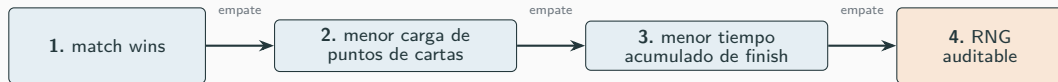
Problema: en JumpIn y ChallengeUno compiten varios jugadores; por orden de llegada, el más cercano al server siempre gana.



- RTT medido **por el servidor** vía heartbeats (rolling average) — el reloj del cliente jamás se usa.
- Ventana de carrera: **150 ms** de llegada. Empate dentro de ± 20 ms (o sin RTT medido) → RNG uniforme.
- Todo queda en el log: evento RaceResolved con RTTs y método — auditable, RTTs no expuestos a rivales.
- Misma suposición ($RTT/2$, ruteo simétrico) que la lag compensation de shooters como CS2.

Decisión: desempates para avanzar de ronda

Bo3 por sala de 10 → avanzan los **top 3**. Orden total:



- `finish_timestamp` con **reloj monotónico del server**; sin podio $\Rightarrow$ contribuye el timeout de 20 min.
- La semilla del RNG sale del match y queda registrada en `MatchCompleted`.
- Forfeits: **debajo de todos los activos**, siempre. Sala con ≤ 3 activos $\Rightarrow$ avanzan todos.

Cada criterio mide algo más fino y siempre disponible: resultado $\rightarrow$ calidad $\rightarrow$ velocidad $\rightarrow$ azar auditable.

Decisión: Redis — un rol distinto por contexto

Uso	Mecanismo
Timers (45 s / 5 s / 60 s / 20 min)	TTL + keyspace notif.
Invalidación de sesión	Pub/Sub → gateway
Fan-out a espectadores	Streams (XREAD)
Cache de sesión (JWT)	cache-aside, TTL 60 s
Rate limiting	INCR + ZSET sliding
Leaderboards Elo	Sorted Sets
Idempotencia / locks	SET NX PX

Dos instancias, políticas opuestas:

redis-timer = noeviction (un timer no se puede perder)

redis-general = cache/fan-out.

Timers: alternativas

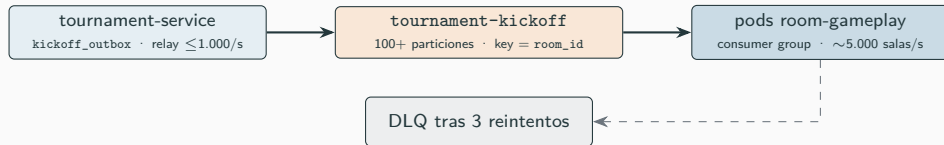
- ✗ in-process: no sobrevive un crash
- ✗ tabla PG + polling: 1 s de jitter en una ventana de 5 s
- ✗ Kafka diferido: sin delay nativo, cancelar es complejo
- ✓ Redis TTL: precisión ms, crash-safe, cancelación O(1)

Doble red de seguridad

- **Token fence**: UUID en la key y en el estado PG
→ expiración duplicada = no-op
- Sweeps de reconciliación (challenge cada 2 s) si se pierde la notificación

Principio: Redis nunca es fuente de verdad — es fast-path; la DB siempre tiene el fallback durable.

Decisión: el surge de kickoff (100 K salas en ≤ 120 s)



Alternativa evaluada

- ✗ Work queue en PG + workers HTTP:
backpressure artesanal, retry storms,
el endpoint HTTP como cuello de botella

Por qué Kafka fan-out

- El **consumer lag** es backpressure natural y observable
- Productor y consumers escalan por separado
- Room IDs **UUID5 determinísticos** → reintentos idempotentes

Alcance honesto: qué corre y qué no

- **analytics-service no desplegado** (ADR-009): el burst ya lo absorben 4 consumer groups independientes; sus read models duplican ranking / tournament / spectator.
Alternativas evaluadas: implementarlo completo ✗ · versión token ✗ · plegarlo en ranking ✗. Pérdida real declarada: stats históricas por jugador.
- **outbox-relay embebido** en cada servicio: ahí vive la frontera transaccional.
- **api-gateway con 1 réplica**: registro de WS in-process; remediación documentada = fan-out Redis Pub/Sub (el mismo mecanismo de la invalidación de sesión).
- **Nodo único**: 1 Postgres (7 schemas, no $\times 16$ shards), 1 broker — scale-down documentado, mismos code paths y contratos.

Decisiones registradas en ADRs, no trabajo inconcluso.

Observabilidad: RED + 6 métricas de negocio

Tres pilares, desplegados

- **Prometheus:** RED por servicio (P99 < 200 ms)
- **Grafana:** dashboard *UnoArena* — *Live*, provisionado como código
- **Jaeger:** la traza **cruza Kafka** vía traceparent en la fila de outbox

Logs JSON estructurados con `correlation_id`
a través de HTTP, WS y headers Kafka.

Métricas de negocio (se piden 3, hay 6)

- `uno_game_events_total` — 1 hook en el log = todo el dominio
- `uno_games_active` — gauge que se auto-repara (lee PG)
- `uno_matchmaking_queue_depth` — sube y cae a 0 al formar sala
- `uno_elo_updates_total` — casual / torneo / **reversión**
- `uno_websocket_connections_active` — jugador vs espectador
- `uno_rate_limit_hits` + `uno_session_events` — abuso y sesión única

Criterio: métricas sobre *el juego*, no sobre el proceso — lo que miraría un operador de torneo.

Demo: lo que van a ver ahora

1. **Despliegue desde cluster vacío** — `deploy-all.sh` (~2 min):
kind → 7 imágenes side-load → infra (PG, Redis ×2, Redpanda + tópicos) → observabilidad → 7 servicios
2. **Partida casual 1v1** — dos terminales interactivas:
play / draw / uno / challenge, WILD con color, 409 stale + reconciliación
3. **Espectador en vivo** — tercera terminal: conteos sí, manos jamás
4. **Sesión única** — segundo login cierra el WS con código 4401 (push, no polling)
5. **Evidencia** — Grafana (negocio + RED), Jaeger (traza que cruza Kafka),
psql: log inmutable y Elo que se movió
6. **Torneo Bo3 con 4 bots** → campeón + Elo de torneo (si el tiempo acompaña)

¡Muchas gracias!